ean universidad

acreditada
en alta calidad
res. N° 023654 del Mineducación
10/12/21, vigencia 10/12/27

FUNDAMENTOS DE PYTHON Y PANDAS

Sesión 1 · Python para Procesamiento, Calidad y Automatización de Datos

Del Excel al DataFrame

Jawy Andrés Romero Pinto · Martes 3 de noviembre de 2026 · 08:00 – 10:00 a. m.

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Acreditaciones:

ean universidad

PARA ARRANCAR

¿Vale la pena estudiar programación en Python en 2026 y 2027?



Responde en el chat de Teams: **Sí · No · Depende**
y en una sola frase, por qué.

Lenguaje compilado vs. interpretado

Dos formas de convertir tu código en algo que la máquina pueda ejecutar

Compilado

C, C++, Rust, Go

Código fuente

Lo que escribe el programador.

Compilador

Traduce TODO el programa, una vez, antes de ejecutar.

Ejecutable

Lenguaje de máquina (.exe) para un sistema operativo.

CPU

Lo ejecuta directo: muy rápido.

Interpretado

Python, JavaScript, R

Código .py

Lo que escribes en Colab.

Intérprete

Lee y ejecuta instrucción por instrucción, en el momento.

CPU

Ves el resultado de inmediato.

Compilado: máxima velocidad de ejecución · los errores aparecen al compilar · hay que compilar para cada sistema operativo.

Interpretado: máxima velocidad de desarrollo · pruebas al instante (notebooks) · el mismo código corre en Windows, Mac, Linux y la nube.

¿Por qué decimos que Python es interpretado?

Por dentro: compila a bytecode y una máquina virtual lo ejecuta

[10]

```python
import dis

def aporte(ibc):
    return ibc * 0.16

# Python "compila" a bytecode y el intérprete lo ejecuta
dis.dis(aporte)
```

```
  3           RESUME                   0

  4           LOAD_FAST_BORROW         0 (ibc)
              LOAD_CONST               0 (0.16)
              BINARY_OP                5 (*)
              RETURN_VALUE
```

Qué observar

- CPython traduce tu `.py` a **bytecode** (los `.pyc`).
- La **máquina virtual** de Python ejecuta esas instrucciones una a una.
- No genera un `.exe`: necesitas el intérprete para correrlo.

¿Por qué lo diseñaron así? Tipos dinámicos y ejecución inmediata: escribes, ejecutas y ves el resultado. Ideal para **explorar datos**.

¿Interpretado = lento? Sí… y no

El truco: Python dirige, C hace el trabajo pesado

[11]

```python
import time
import numpy as np

numeros = list(range(10_000_000))
t0 = time.perf_counter()
total = 0
for n in numeros:            # Python puro: uno por uno
    total += n
t_py = time.perf_counter() - t0

arreglo = np.arange(10_000_000)
t0 = time.perf_counter()
total_np = arreglo.sum()     # NumPy: el bucle corre en C
t_np = time.perf_counter() - t0

print(f"Python puro: {t_py:.3f} s")
print(f"NumPy:       {t_np:.4f} s  → {t_py / t_np:,.0f} veces más rápido")
```

```
Python puro: 0.501 s
NumPy:       0.0047 s  → 107 veces más rápido
```

Qué observar

- El bucle de Python revisa tipos **en cada vuelta**.
- NumPy y pandas corren el bucle en **C compilado**.
- Medido de verdad: **~100 veces** más rápido.

Lo mejor de los dos mundos: **escribes en Python** (fácil) y **ejecutas en C** (rápido). En pandas casi nunca necesitas un `for`.

¿Por qué Python para análisis de datos?

Seis razones por las que se volvió el estándar

Se lee casi como inglés

Sintaxis limpia: menos símbolos, más intención. Fácil de leer y revisar.

Ecosistema completo

pandas, NumPy, matplotlib, scikit-learn, Polars: una librería para cada paso.

Notebooks

Código, resultados, gráficos y explicación en un solo documento (Jupyter, Colab).

Habla con todo

CSV, Excel, SQL, APIs, la nube y PDF: lee y escribe casi cualquier fuente.

Gratis y abierto

Sin licencias: en tu equipo, en un servidor o en el navegador.

Comunidad gigante

Si te sale un error, alguien ya lo resolvió y lo publicó.

Python es el **n.º 1 del índice TIOBE desde 2021** y en 2024 superó a JavaScript como el lenguaje **más usado en GitHub** (Octoverse 2024).

Python y los modelos de lenguaje (LLM)

ChatGPT, Claude, Gemini o Llama se investigan, entrenan y usan con herramientas de Python



Tu aplicación o agente

SDK de Anthropic y OpenAI, LangChain, MCP: Python primero.



Modelos y datos

Hugging Face: Transformers, Datasets, Tokenizers.



Frameworks de entrenamiento

PyTorch (Meta), JAX (Google), TensorFlow.



Cálculo en GPU

C++ y CUDA: el motor que Python orquesta.

PyTorch (2017) llevó el aprendizaje profundo a Python y hoy domina la investigación en IA.

El código de los artículos científicos de IA se publica, casi siempre, **en Python**.

Python es el volante, no el motor: escribes pocas líneas y miles de GPU hacen el cálculo.

Tecnologías que mueven el mundo… y usan Python

Está en todos los sistemas de hoy: web, streaming, ciencia, banca y hasta en Excel

Instagram

Corre sobre Django, framework web de Python.

Netflix

Recomendaciones, datos, automatización y seguridad.

Spotify

Creó Luigi para orquestar sus flujos de datos.

Apache Airflow

Nació en Airbnb; hoy orquesta datos en bancos y aseguradoras.

Agujero negro (2019)

La primera imagen se reconstruyó con librerías de Python.

Ondas gravitacionales

LIGO (Nobel 2017) analiza sus señales con Python.

Microsoft Excel

Python en Excel: fórmulas =PY() dentro de la hoja.

Banca

JPMorgan usa Python en Athena, su plataforma de riesgo.

Viene **preinstalado en Linux** (la mayoría de los servidores del mundo), corre en **AWS, Azure y Google Cloud** y se usa desde **Power BI y Excel**. Si una organización tiene datos, casi seguro tiene Python.

¿Por qué Python es el mejor lenguaje para IA?

No es el más rápido: es el que mejor une las piezas



Lenguaje pegamento

Une piezas escritas en C, C++ y CUDA con una sintaxis simple.



Experimentar rápido

En IA se prueban cientos de ideas: un notebook lo hace en minutos.



Todo el ecosistema

scikit-learn, PyTorch, TensorFlow, Hugging Face, LangChain.



La IA lo escribe bien

Es de los lenguajes con más código público: los asistentes lo dominan.

Tarea de IA	Librería de Python	Ejemplo en una entidad
Aprendizaje automático	scikit-learn	Predecir qué afiliados dejarán de aportar
Lenguaje natural / LLM	Transformers, SDK de Claude	Clasificar PQRS y resumir solicitudes
Agentes	LangChain, MCP, Claude Agent SDK	Revisar la calidad de un archivo y reportar

De 1991 a hoy: cómo ha evolucionado Python

35 años: de proyecto personal al lenguaje más usado del mundo

1991 · 0.9

Guido van Rossum lo publica (CWI, Países Bajos).

2000 · 2.0

List comprehensions y recolector de basura.

2008 · 3.0

Texto Unicode por defecto; rompe compatibilidad.

2016 · 3.6

Llegan las f-strings: `f"{x}"`.

2020 · Fin de 2.x

Python 2 deja de tener soporte (1 de enero).

2022 · 3.11

Faster CPython: 1,25× más rápido en promedio.

2024 · 3.13

Modo sin GIL y compilador JIT, experimentales.

2025 · 3.14

Sin GIL con soporte oficial; t-strings.

2026 · 3.15

Imports diferidos (lazy) y UTF-8 por defecto.

Lección para quien trabaja con datos: pasar de Python 2 a 3 tomó 12 años. El código también envejece: por eso cada notebook del curso indica qué versiones usa.

Rendimiento, velocidad y seguridad: de Python 3 a 3.14

El mismo código que escribes hoy corre más rápido y más seguro con cada versión

Velocidad

- **3.11** · Faster CPython: hasta 60 % más rápido, **1,25×** en promedio.
- **3.12** · Mejores mensajes de error y f-strings más flexibles.
- **3.13** · Primer compilador **JIT** (experimental).
- **3.14** · Nuevo intérprete con tail calls: unos puntos más.

Paralelismo (adiós al GIL)

- **Antes** · El GIL deja correr un solo hilo de Python a la vez.
- **3.12** · Un GIL por subintérprete (PEP 684).
- **3.13** · Versión experimental **sin GIL** (PEP 703).
- **3.14** · Sin GIL con **soporte oficial** y subintérpretes en la librería estándar.

Seguridad

- **3.3** · Hash aleatorio por defecto contra ataques de denegación de servicio.
- **3.6** · Módulo `secrets` para tokens y contraseñas seguros.
- **3.8** · Audit hooks: registrar qué hace el código (PEP 578).
- **3.13** · Se retiran 19 módulos obsoletos (PEP 594).
- **3.14** · HMAC con código **verificado formalmente** y firmas Sigstore.

GIL (Global Interpreter Lock): el candado que solo deja ejecutar un hilo de Python a la vez. Quitarlo permite usar **todos los núcleos** del procesador en un mismo programa.

Mi respuesta: sí vale la pena, y con la IA más que antes

Volvamos a la pregunta del inicio

«La IA puede escribir el código. **Tú respondes por lo que hace.**»

Jawy Romero · Delego la programación en Claude Code, y por eso mismo necesito saber revisarla.

Revisar exige entender

Si un asistente escribe el script, alguien debe leerlo antes de usarlo. Sin bases, solo queda confiar a ciegas.

Los errores no avisan

El código puede correr sin fallar y entregar un número equivocado. Lo vemos en la siguiente diapositiva.

Pedir bien es programar

Describir datos, reglas y casos especiales con precisión es pensamiento computacional.

Tú firmas, no la IA

Con datos de afiliados (Habeas Data, auditoría), un script revisado es evidencia; uno que no entiendes es un riesgo.

En 2026 y 2027 no compites contra la IA: **compites con quien sabe usarla y revisarla.** Para eso basta con dominar las bases, y eso es lo que haremos en estas 5 sesiones.

Ejemplo: código que corre… y se equivoca

Le pedimos a un asistente contar los afiliados activos

[12]

```python
# Pedido a un asistente de IA: "¿cuántos afiliados activos hay?"
activos = afiliados[afiliados["estado"] == "Activo"]
print("Respuesta de la IA:", len(activos), "activos")

# Revisión humana: ¿qué valores trae de verdad la columna?
estado = afiliados["estado"].str.strip().str.lower()
print("Revisando el dato:", (estado == "activo").sum(), "activos")
estado.value_counts()
```

```
Respuesta de la IA: 1898 activos
Revisando el dato: 1955 activos
estado
activo        1955
inactivo       583
pensionado     565
pensionada      17
Name: count, dtype: int64
```

Qué observar

- La IA filtró `== "Activo"` **exacto**.
- Dejó por fuera **57 afiliados**: «activo», «Activo » y «ACTIVO».
- Sin error y sin aviso: solo un número equivocado.

Para esto vale la pena saber Python: no para competirle a la IA escribiendo código, sino para **darte cuenta cuando se equivoca**.

Sobre el docente

JR

Jawy Andrés Romero Pinto

Ingeniero de Ciberseguridad

Docente · Universidad Ean — Educación Continua

En Ean también dicto

- Ciber-recuperación ante ransomware
- FinOps: gestión de costos en la nube
- Python para procesamiento de datos

Cómo trabajamos

- 100 % práctico, en Google Colab
- Un mismo caso de principio a fin
- Automatizar lo que hoy haces a mano

Dónde encontrarme

- Chat de Teams durante la clase
- Grupo de WhatsApp del programa
- Material: repositorio en GitHub

Tu turno, en el chat de Teams: tu nombre · tu área en Colpensiones · ¿qué tarea con datos repites cada mes y te gustaría automatizar?

ean universidad

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Bienvenida: reglas de juego

Para que las 10 horas rindan

80 % de asistencia mínima para certificar: máximo 2 horas de inasistencia en todo el programa.

Conéctate a Teams 5 minutos antes. A las 08:00 a. m. en punto arrancamos.

Canal oficial: grupo de WhatsApp. Material: GitHub + carpeta de OneDrive.

Diligencia el formulario de Habeas Data: es requisito para el certificado.

Trabajamos en Google Colab: solo necesitas navegador y una cuenta Google.

Si te atascas, escríbelo en el chat. Nadie se queda atrás.

El programa en 5 sesiones

10 horas · un mismo caso de principio a fin: un fondo de pensiones ficticio

Sesión	Fecha	Tema	Pregunta que respondemos	En pandas
1 · Hoy	Mar 3 nov	Fundamentos de Python y pandas	¿Qué tengo en mis datos?	read_csv · head · info
2	Mié 4 nov	Limpieza y transformación	¿Cómo dejo los datos usables?	str · to_datetime · merge
3	Jue 5 nov	Calidad de datos	¿Puedo confiar en mis datos?	isna · duplicated · reglas
4	Vie 6 nov	Automatización	¿Cómo hago que se repita solo?	def · bucles · scripts
5	Lun 9 nov	Proyecto integrador	¿Puedo construir el flujo completo?	todo junto

Cada sesión: **~40 min** de conceptos con demo en vivo + **~60 min** de laboratorio en Colab + cierre y reto.

Agenda de hoy

Martes 3 de noviembre · 08:00 – 10:00 a. m.

08:00	**La gran pregunta**	¿Vale la pena Python en 2026? Lenguaje, datos e IA.
08:15	**Bienvenida**	Presentación, reglas de juego y el programa.
08:20	**Python en tu trabajo**	Excel no desaparece: Python lo complementa.
08:24	**Las herramientas**	Python, pandas, Google Colab y GitHub, en vivo.
08:32	**Python esencial**	Variables, listas, diccionarios, funciones y texto.
08:41	**pandas**	Leer, explorar, contar, filtrar y resumir datos reales.
08:55	**Laboratorio**	Diagnóstico del archivo de afiliados en Colab (60 min).
09:55	**Cierre y reto**	Lo que te llevas y la tarea para mañana.

ean universidad

01

Python en tu trabajo

Si lo haces más de dos veces a mano, merece un script.

¿Por qué Python si ya tengo Excel?

Excel no desaparece: Python lo complementa cuando el trabajo crece

Excel brilla cuando…

- Los datos caben en una hoja (≤ 1.048.576 filas).
- El análisis es puntual, de una sola vez.
- Necesitas ver y editar celda a celda.
- Lo usa alguien que no programa.

Python brilla cuando…

- Son muchos archivos o millones de filas.
- El proceso se repite cada día, semana o mes.
- Cada paso debe quedar escrito y ser auditable.
- Quieres validar calidad y generar reportes solo.

Un script es una **receta**: se puede leer, revisar, auditar y volver a ejecutar con los datos del próximo mes. Hoy: **9 archivos de aportes** a mano en Excel ≈ una mañana. En Python ≈ **1 segundo** (Sesión 4).

ean universidad

02

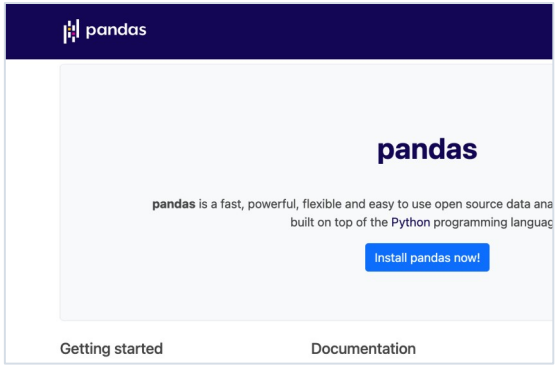
Las herramientas del curso

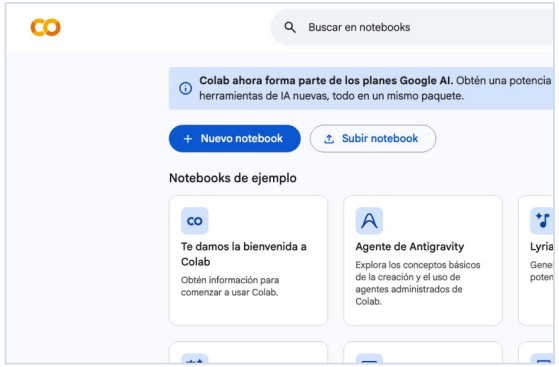
Todo en el navegador: nada que instalar en tu equipo.

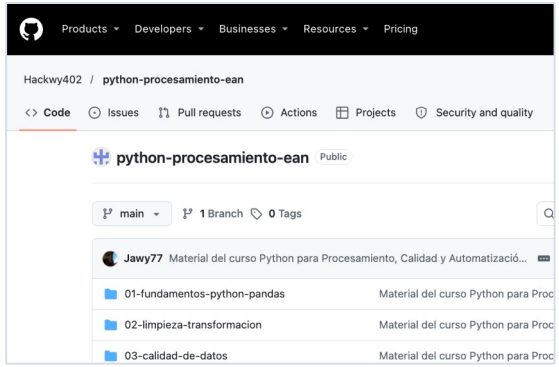
Tus 4 herramientas

Capturas reales de cada sitio









Python

El lenguaje. Gratuito y el más usado en datos.

python.org

pandas

La librería para tablas: el «Excel» de Python.

pandas.pydata.org

Google Colab

Notebooks en el navegador, con pandas ya instalado.

colab.research.google.com

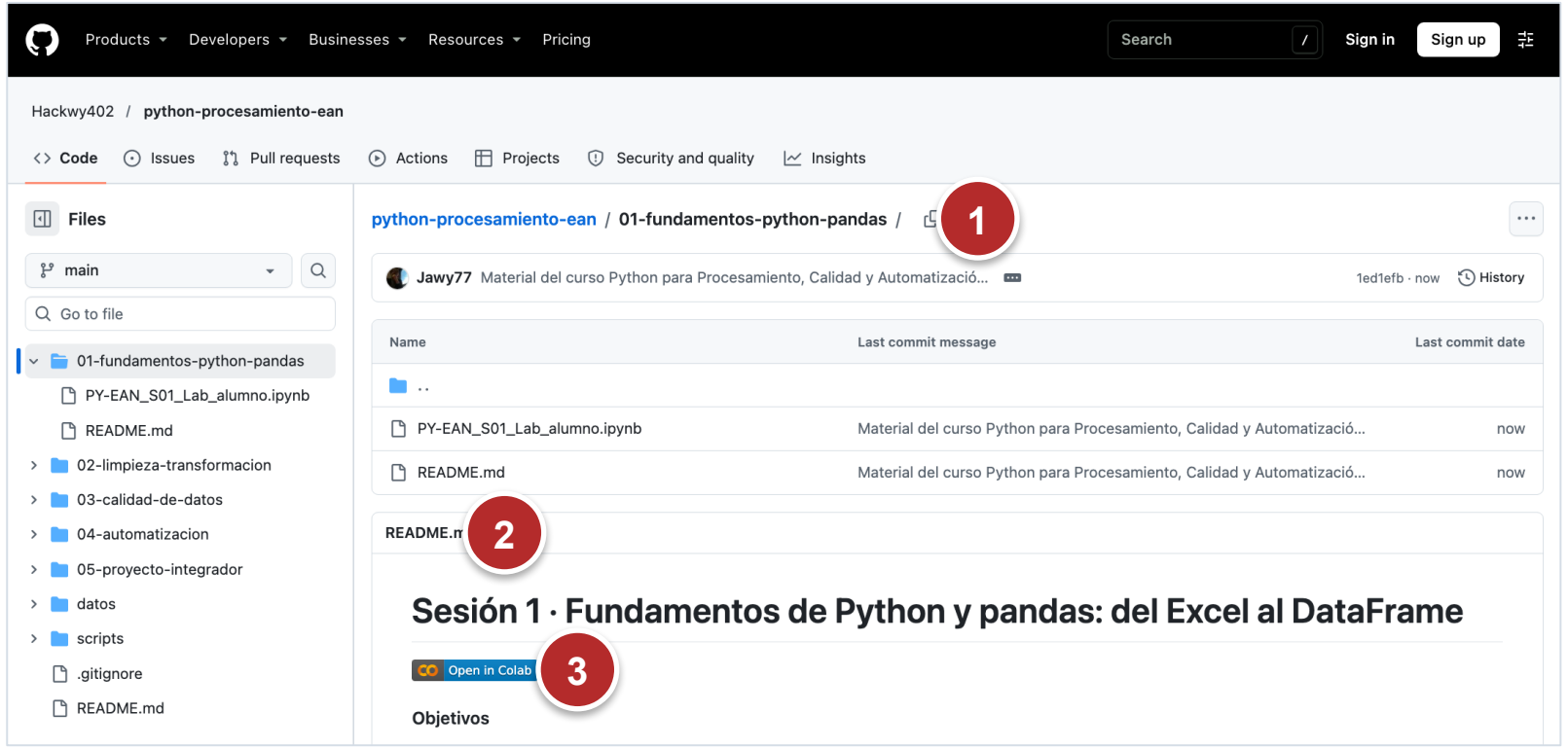
GitHub

Donde vive el material del curso: datos y notebooks.

github.com/Hackwy402/python-procesamiento-ean

El material del curso está en GitHub

github.com/Hackwy402/python-procesamiento-ean

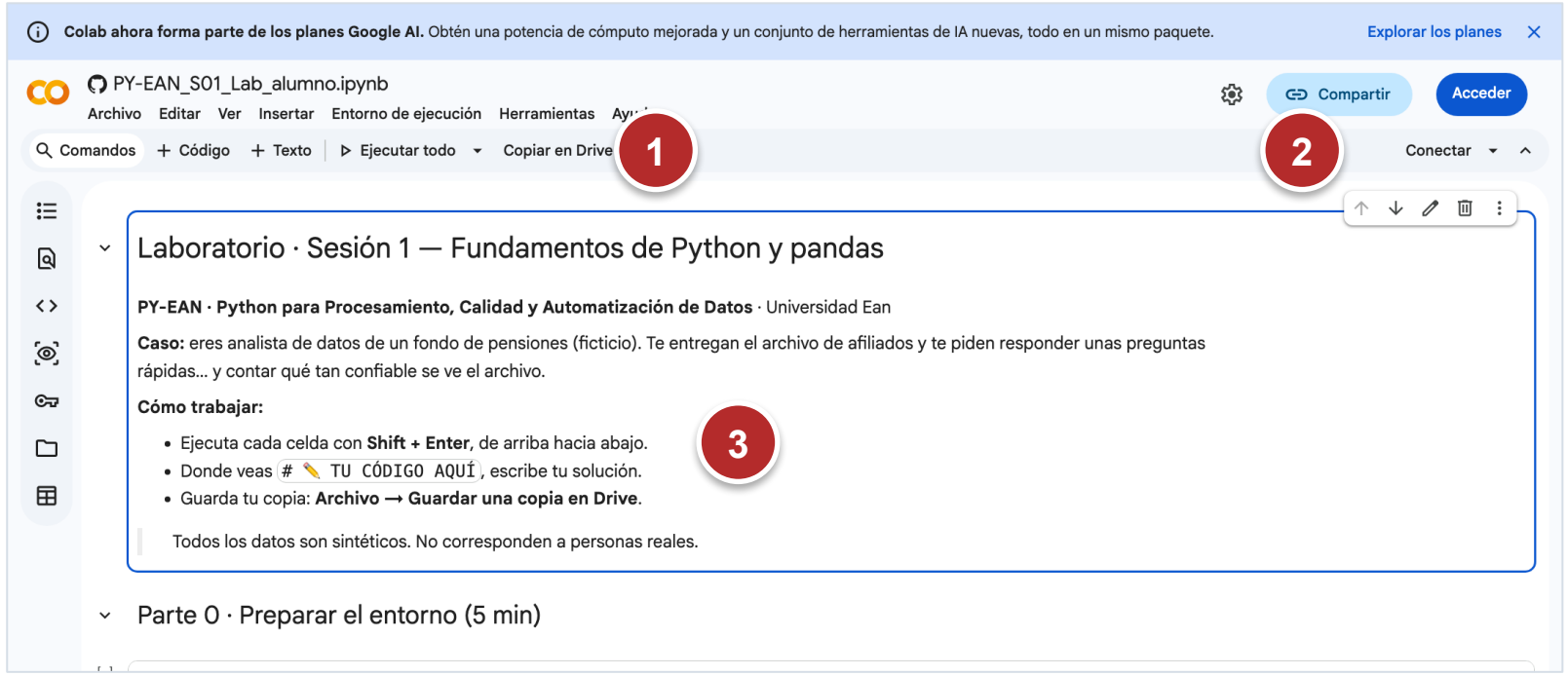


1. **Carpeta de la sesión:** `01-fundamentos-python-pandas`.
2. **README** de la sesión: objetivos, datos y duración.
3. Botón **Open in Colab**: abre el notebook listo para trabajar.

> Los datos se leen **directo desde GitHub**: no tienes que descargar nada.

Google Colab: tu entorno de trabajo

El notebook de hoy, abierto desde GitHub



1. **Copiar en Drive**: tu propia copia, donde sí puedes guardar.
2. **Conectar**: Colab te asigna una máquina en la nube.
3. **Shift + Enter** en cada celda: el resultado sale debajo.

Las celdas corren **en el orden en que tú las ejecutas**. ¿`NameError`? Te saltaste una celda: **Entorno de ejecución → Ejecutar todas**.

ean universidad

03

Python esencial para datos

Lo mínimo que necesitas para leer y escribir código de procesamiento.

Variables y tipos

Una variable es una etiqueta que apunta a un valor

[1]
```
nombre = "María Gómez"          # str   → texto
edad = 45                       # int   → entero
salario = 2_850_000.0           # float → decimal
es_pensionado = False           # bool  → verdadero/falso

aporte = salario * 0.16
print(nombre, "aporta", aporte)
print(type(salario), type(edad))
print(f"{nombre} tiene {edad} años y aporta ${aporte:,.0f}")
```

```
María Gómez aporta 456000.0
<class 'float'> <class 'int'>
María Gómez tiene 45 años y aporta $456,000
```

Qué observar

- `str` texto, `int` entero, `float` decimal, `bool` sí/no.
- `type()` te dice qué es cada cosa.
- **f-strings**: `f"{valor:,.0f}"` formatea miles.

"1.300.000" (texto) no es lo mismo que **1300000** (número). Esto nos va a perseguir todo el curso.

Listas y diccionarios

Las estructuras que usarás todo el tiempo

[2]
```python
# Lista: colección ordenada (como una columna)
ciudades = ["Bogotá", "Medellín", "Cali"]
ciudades.append("Pereira")
print(ciudades[0], "|", len(ciudades), "ciudades")

# Diccionario: clave → valor (como una fila)
afiliado = {"id": "AF100026", "ciudad": "Medellín", "salario": 1_819_000}
afiliado["estado"] = "Activo"
print(afiliado["ciudad"])
print(afiliado)
```

```
Bogotá | 4 ciudades
Medellín
{'id': 'AF100026', 'ciudad': 'Medellín', 'salario': 1819000, 'estado': 'Activo'}
```

Qué observar

- **Lista** `[ ]`: colección ordenada ≈ una columna.
- Se cuenta **desde 0**: `ciudades[0]`.
- **Diccionario** `{ }`: clave → valor ≈ una fila.

Una **lista de diccionarios** ya es una tabla. pandas la convierte en DataFrame con una línea.

Decisiones, ciclos y funciones

La lógica que luego vamos a automatizar

[3]

```python
SMMLV = 1_750_905

def calcular_aporte(ibc, tasa=0.16):
    """Devuelve el aporte a pensión para un IBC dado."""
    if ibc < SMMLV:
        print("  AVISO: IBC por debajo del salario mínimo")
    return round(ibc * tasa)

for ibc in [1_200_000, 2_000_000, 5_500_000]:
    print(f"IBC {ibc:>10,} → aporte {calcular_aporte(ibc):>9,}")
```

```
  AVISO: IBC por debajo del salario mínimo
IBC  1,200,000 → aporte   192,000
IBC  2,000,000 → aporte   320,000
IBC  5,500,000 → aporte   880,000
```

Qué observar

- `def` empaqueta pasos que se repiten.
- `if` decide; `for` recorre una lista.
- `tasa=0.16` es un valor por defecto.

La **indentación** (4 espacios) no es decoración: define qué está dentro del `if`, del `for` y de la función.

Texto: el primer paso de la limpieza

Métodos de str que vas a usar a diario

[4]

```
nombre_crudo = "  edwin HÉCTOR  "
print(repr(nombre_crudo.strip().title()))

ciudades = ["Bogotá", "BOGOTÁ ", " bogotá", "Medellín"]
print(len(set(ciudades)), "ciudades distintas… ¿seguro?")

limpias = [c.strip().title() for c in ciudades]
print(limpias, "→", len(set(limpias)), "distintas")

salario_txt = "1.819.000"
print(int(salario_txt.replace(".", "")) * 0.16)
```

```
'Edwin Héctor'
4 ciudades distintas… ¿seguro?
['Bogotá', 'Bogotá', 'Bogotá', 'Medellín'] → 2 distintas
291040.0
```

Qué observar

- `.strip()` quita espacios; `.title()` / `.upper()` normalizan.
- `set()` deja los valores únicos.
- `.replace(".", "")` arregla puntos de miles.

Para el computador **"BOGOTÁ "** y **"Bogotá"** son ciudades distintas. Mañana lo resolvemos en toda la columna.

ean universidad



04

pandas: tu Excel con superpoderes

Leer, explorar, contar, filtrar y resumir: con el archivo real de afiliados.

De Excel a pandas

Los mismos conceptos, con otro nombre

En Excel	En pandas	Ejemplo
Libro / hoja	DataFrame	`afiliados = pd.read_csv(...)`
Columna (A, B, C…)	Series	`afiliados["ciudad"]`
Número de fila	Índice (index)	`afiliados.loc[0]`
Filtro	Máscara booleana	`afiliados[afiliados["estado"] == "Activo"]`
CONTAR.SI / tabla dinámica	value_counts / groupby	`afiliados["ciudad"].value_counts()`
BUSCARV	merge (Sesión 2)	`afiliados.merge(oficinas, on="ciudad")`

`import pandas as pd` → por convención siempre se importa con el alias **pd**.

Leer y mirar los datos

read_csv + head: tus primeras dos líneas de pandas

[5]

```python
import pandas as pd

afiliados = pd.read_csv("datos/afiliados.csv")
print(afiliados.shape)          # (filas, columnas)

afiliados[["id_afiliado", "nombres", "ciudad",
           "salario_base", "estado"]].head()
```

```
(3120, 14)
```

	id_afiliado	nombres	ciudad	salario_base	estado
0	AF100026	PATRICIA	Medellín	1819000	Activo
1	AF100492	Andrea	Cartagena	1760000	Activo
2	AF101579	edwin héctor	Manizales	8985000	Activo
3	AF102295	Andrés	Bogotá D.C.	3.549.000	Activo
4	AF100510	Óscar Fernando	Bogotá D.C.	5522000	Activo

Qué observar

- **3.120 filas × 14 columnas** en una línea.
- `.head()` muestra las 5 primeras.
- Fila 3: salario **3.549.000** (texto con puntos).

Excel:
`pd.read_excel("datos/oficinas.xlsx", sheet_name="ciudades")`.
Mismo patrón, otro formato.

info(): la radiografía del archivo

El comando más subestimado de pandas

[6]
```
afiliados.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3120 entries, 0 to 3119
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   id_afiliado       3120 non-null   str
 1   tipo_documento    3120 non-null   str
 2   numero_documento  3105 non-null   str
 3   nombres           3120 non-null   str
 4   apellidos         3120 non-null   str
 5   fecha_nacimiento  3120 non-null   str
 6   genero            3056 non-null   str
 7   ciudad            3120 non-null   str
 8   departamento      3120 non-null   str
 9   fecha_afiliacion  3120 non-null   str
 10  salario_base      3088 non-null   str
 11  email             3105 non-null   str
 12  telefono          3098 non-null   float64
 13  estado            3120 non-null   str
dtypes: float64(1), str(13)
memory usage: 341.4 KB
```

Qué observar

- **Non-Null Count**: vacíos por columna.
- `salario_base` es **str**: hay texto mezclado.
- `telefono` es **float**: tiene vacíos.

Si una columna de números aparece como texto, **hay basura dentro**. Ese es tu primer hallazgo de calidad.

Contar: value_counts()

La tabla dinámica en una línea

[7]

```
# ¿Cuántos afiliados por ciudad? (top 8)
afiliados["ciudad"].value_counts().head(8)
```

```
ciudad
Bogotá D.C.      825
Medellín         347
Cali             292
Barranquilla     221
Cartagena        152
Bucaramanga      151
Pereira          112
Cúcuta           105
Name: count, dtype: int64
```

Qué observar

- Ordena de mayor a menor.
- `.head(8)` = top 8.
- `normalize=True` → porcentajes.

Bogotá D.C. 825… pero en total hay **39 ciudades distintas** para 15 reales. Bogotá aparece de **7 formas**.

Filtrar: preguntas de negocio en código

Máscaras booleanas con & (y) y | (o)

[8]
```python
# Mujeres activas en Medellín
filtro = ((afiliados["ciudad"] == "Medellín")
          & (afiliados["estado"] == "Activo")
          & (afiliados["genero"] == "F"))
print("Coinciden:", filtro.sum())

cols = ["nombres", "apellidos", "fecha_afiliacion"]
afiliados.loc[filtro, cols].head(4)
```

```
Coinciden: 115
```

	nombres	apellidos	fecha_afiliacion
0	PATRICIA	Ortiz Ortiz	2007-03-17
39	Juliana	Zapata Hernández	2022-02-18
44	María Sandra	Zapata Ramírez	2018-06-10
53	Andrea	Gómez Muñoz	1996-07-25

Qué observar

- Cada condición **entre paréntesis**.
- `&` y `|`, **no** `and` / `or`.
- `.loc[filas, columnas]` elige ambas.

Solo cuenta lo escrito **exacto**: "Medellin", "f" o "Femenino" quedan por fuera. El filtro es tan bueno como los datos.

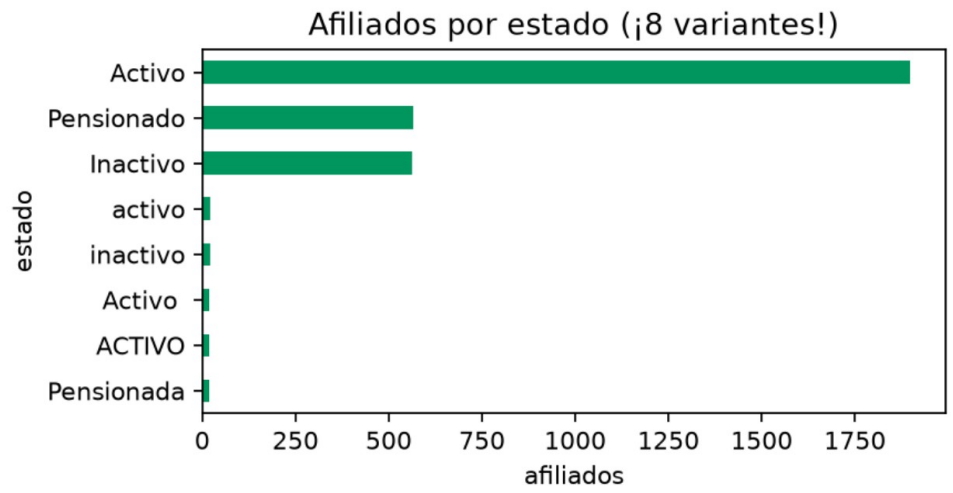
Resumir y graficar

to_numeric + value_counts + plot

[9]
```
sal = pd.to_numeric(afiliados["salario_base"],
                    errors="coerce")
print("Vacíos o con puntos:", sal.isna().sum())

# ¿Cuántos afiliados hay por estado?
conteo = afiliados["estado"].value_counts()
conteo.sort_values().plot.barh(
    color="#00965E", xlabel="afiliados",
    title="Afiliados por estado (¡8 variantes!)")
```

```
Vacíos o con puntos: 95
```



Qué observar

- `errors="coerce"` deja **NaN** lo que no convierte.
- `.plot.barh()` grafica sin salir de pandas.
- **8 «estados»** donde debería haber 3.

Activo, ACTIVO, activo y "Activo " son el mismo estado. **Mañana (Sesión 2)** los unificamos con código.

LAB

60 min

Laboratorio · Sesión 1

Abre **PY-EAN_S01_Lab_alumno.ipynb** desde GitHub → **Open in Colab** → **Copiar en Drive**.

Exploras `afiliados.csv`, respondes preguntas de negocio y anotas los problemas de calidad que veas.

Reto para mañana: tus 5 problemas de calidad más graves. En la Sesión 2 los corregimos con código.

ean universidad

acreditada
en alta calidad
res. Nº 023654 del Mineducación
10/12/21, vigencia 10/12/27



www.universidadean.edu.co

Universidad Ean: SNIES 2812 Personería Jurídica Res. 2898 del Minjusticia - 16/05/69 - Vigilada Mineducación.

Ean Legacy: Carrera 11 No. 78-47 - Fundadores: Calle 79 # 11 - 45 - Av. Chile: Calle 71 # 9 - 84 Bogotá D.C., Cundinamarca, Colombia, Suramérica.

Línea gratuita nacional: +57-01-8000-931000
Centro de contacto en Bogotá: +(57-601) 593 6161 - +(57-601) 593 6464
E-mail: informacion@universidadean.edu.co

© 2024 Todos los derechos reservados - Universidad Ean